Áp Dụng Clean Architecture & DI Trong Flutter

TRƯỜNG ĐẠI HỌC BÁCH KHOA — KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Chủ đề 04: Báo cáo Tuần 01 — Môn: Lập trình Đa nền tảng

SVTH: Huỳnh Thiện Lộc – **MSSV:** 106230052
GVHD: TS. Nguyễn Duy Nhật Viễn

Đà Nẵng, 2026

Nội dung báo cáo

1. Phần I: Bài Toán Thực Tế & Thiết Kế 3 Tầng
   - Bài toán & Tiếp cận kiến trúc
   - Trách nhiệm của 3 tầng kiến trúc
   - Nguyên lý Dependency Inversion (DIP)
2. Phần II: Dependency Injection với get_it
   - Đăng ký phụ thuộc trong injection_container.dart
   - Khởi tạo hệ thống trong main.dart
3. Phần III: Demo Thực Tế & Kết Quả Kiểm Thử
   - Data Flow & Giao diện thực tế
   - Kiểm thử tự động tầng Domain với Mocktail
   - Tài liệu tham khảo cụ thể

Phân công công việc (WBS)

TT	HỌ VÀ TÊN	NỘI DUNG ĐẢM NHẬN	TỶ LỆ
1	Huỳnh Thiện Lộc (106230052)	Thiết kế 3 tầng Clean Arch, triển khai DI (`get_it`), lập trình BLoC và viết Unit Test Mocktail.	100%

Phần I: Bài Toán Thực Tế & Thiết Kế 3 Tầng

Bài toán & Tiếp cận kiến trúc

- **Yêu cầu:** Đọc và hiển thị danh sách bài viết từ REST API (`jsonplaceholder.typicode.com/posts`).

Tight Coupling

- Gọi HTTP trực tiếp trong Widget (`initState`).
- Logic mạng gắn chặt vào UI.
- **Hạn chế:** Không thể Unit Test; đổi API ảnh hưởng trực tiếp đến UI.

Clean Architecture

- **Presentation:** `PostPage` + `PostBloc`
- **Domain:** `Post` + `GetPostsUseCase`
- **Data:** `RemoteDataSource` + `PostRepoImpl`

Trách nhiệm của 3 tầng kiến trúc

Presentation

- **Vai trò:** Hiển thị UI và quản lý trạng thái màn hình.
- **Thành phần:** `PostPage` (Widget), `PostBloc` (State).

Domain (Core)

- **Vai trò:** Nghiệp vụ thuần Dart, độc lập 100% bên ngoài.
- **Thành phần:** `Post` (Entity), `UseCase`, `Repo Interface`.

Data

- **Vai trò:** Gọi REST API và parse dữ liệu sang Model.
- **Thành phần:** `RemoteDataSource`, `PostModel`, `RepoImpl`.

```
// PostBloc (Presentation) chỉ gọi UseCase (Domain), nhận Either và emit State:
final result = await getPostsUseCase();
result.fold((failure) => emit(PostError(message: 'Lỗi tải dữ liệu')),
            (posts) => emit(PostLoaded(posts: posts)));
```

Nguyên lý Dependency Inversion (DIP)

- **Vấn đề:** Nếu UseCase gọi trực tiếp API/Data thì Domain bị phụ thuộc thư viện mạng.
- **Giải pháp:** Domain định nghĩa Interface hợp đồng, Data triển khai, Presentation sử dụng:

```
// 1. Interface tại Domain (Hợp đồng trừu tượng)
abstract class PostRepository {
  Future<Either<Failure, List<Post>>> getPosts();
}
// 2. Data Layer hiện thực Interface
class PostRepositoryImpl implements PostRepository { ... }

// 3. Domain UseCase chỉ nhận Interface qua Constructor Injection
class GetPostsUseCase {
  final PostRepository repository;
  GetPostsUseCase(this.repository);
  Future<Either<Failure, List<Post>>> call() => repository.getPosts();
}
```

Phần II: Dependency Injection với get_it

Đăng ký phụ thuộc trong injection_container.dart

- **Vấn đề:** Tự `new` phụ thuộc trong class gây gắn kết chặt và khó kiểm thử, dễ rò rỉ RAM.
- **Giải pháp:** Đăng ký tập trung bằng `get_it`, phân biệt rõ vòng đời đối tượng:

```
// Stateless: Khởi tạo 1 lần duy nhất trong toàn bộ vòng đời app (tiết kiệm RAM)
sl.registerLazySingleton<http.Client>(() => http.Client());
sl.registerLazySingleton<PostRemoteDataSource>(() =>
PostRemoteDataSourceImpl(client: sl()));
sl.registerLazySingleton<PostRepository>(() => PostRepositoryImpl(remoteDataSource:
sl()));
sl.registerLazySingleton(() => GetPostsUseCase(sl()));

// UI State: Khởi tạo mới theo màn hình, MultiBlocProvider tự close() khi unmount
sl.registerFactory(() => PostBloc(getPostsUseCase: sl()));
```

Khởi tạo hệ thống trong main.dart

- **Khởi tạo WidgetsFlutterBinding:**

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized(); // Native Engine
  await di.initDependencies();                // Service Locator
  runApp(const MyApp());
}
```

- **Quản lý BLoC qua MultiBlocProvider:**

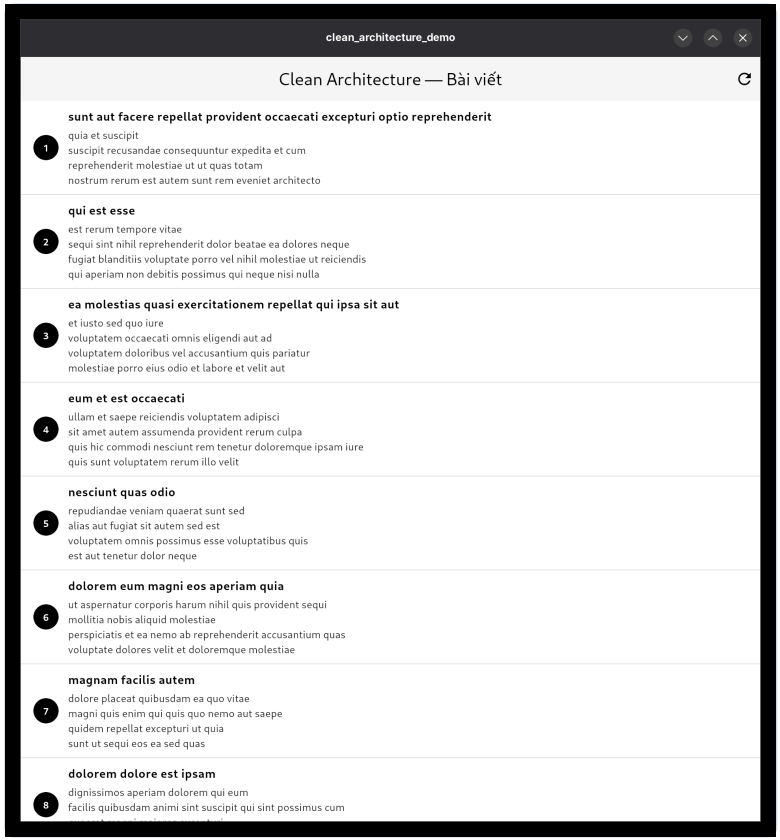
```
MultiBlocProvider(
  providers: [
    BlocProvider<PostBloc>(create: (_) =>
di.sl<PostBloc>()..add(FetchPostsEvent())),
  ],
  child: const MaterialApp(home: PostPage()),
)
```

Phần III: Demo Thực Tế & Kết Quả Kiểm Thử

Data Flow & Giao diện thực tế

REST API ⟶ `RemoteDataSource`
⟶ `PostModel` ⟶ `PostRepoImpl`
⟶ `GetPostsUseCase` ⟶ `PostBloc`

- **Xử lý 3 trạng thái BLoC:**
  - **Loading:** `CircularProgressIndicator`
  - **Loaded:** `ListView.separated` (`List<Post>`)
  - **Error:** Báo lỗi & nút **Thử lại**



Kiểm thử tự động tầng Domain với Mocktail

- **Vấn đề:** Test phụ thuộc mạng sẽ chậm, thiếu ổn định và không chủ động tạo lỗi server.
- **Giải pháp:** Mock interface `PostRepository` bằng `mocktail` để kiểm thử biệt lập:

```
// Success Case: Mock trả về dữ liệu mẫu, kiểm tra UseCase nhận đúng dữ liệu
when(() => mockRepo.getPosts()).thenAnswer((_) async => const Right(tPosts));
final result = await useCase();
expect(result, const Right(tPosts));
verify(() => mockRepo.getPosts()).called(1);

// Failure Case: Mock trả về lỗi server, kiểm tra UseCase đóng gói lỗi an toàn
when(() => mockRepo.getPosts()).thenAnswer((_) async => const
Left(ServerFailure()));
final failureResult = await useCase();
expect(failureResult, const Left(ServerFailure()));
```

- **Kết quả:** Chạy `flutter test` đạt **5/5 tests pass trong 0.1s**, độc lập 100% với mạng.

Tài liệu tham khảo cụ thể

1. **Robert C. Martin (2017)**, *Clean Architecture*, Prentice Hall:
   - **Chương 22 (The Clean Architecture)**: Phân chia 3 tầng và Dependency Rule.
   - **Chương 11 (DIP)**: Nguyên tắc đảo ngược phụ thuộc giữa module cấp cao và cấp thấp.
2. **Reso Coder (2019)**, *Flutter Clean Architecture Proposal*:
   - Thiết kế UseCase và xử lý lỗi Functional với kiểu `Either<Failure, T>`.
3. **Felix Angelov**, *Bloc Library Documentation* (https://bloclibrary.dev):
   - Mô hình State Machine trong Flutter và vòng đời `MultiBlocProvider`.
4. **Thomas Burkhart**, *get_it Service Locator* (https://pub.dev/packages/get_it):
   - Cơ chế Service Locator, phân biệt `registerLazySingleton` và `registerFactory`.